

Лабораторна робота № 2. Перший агент

Мета

Написати програму, яка сама вирішує, які інструменти викликати, щоб відповісти на питання користувача.

Що треба зробити

У вас є три готові інструменти: пошук, довідник і калькулятор.

Ви маєте написати програму, яка працює так:

1. передає моделі питання користувача і описи доступних інструментів;
2. модель відповідає, який інструмент викликати і з якими аргументами;
3. програма викликає цей інструмент і повертає моделі результат;
4. усе повторюється, поки модель не дасть готову відповідь.

Потім запустити цю програму на п'яти питаннях і записати, де вона помилилася.

Хід роботи

Завдання індивідуальне: предметна область, перелік інструментів, системне повідомлення та набір контрольних питань наведено у файлі відповідного варіанта.

Підготовка

Створіть каталог проєкту, віртуальне середовище Python та встановіть бібліотеку `openai`.

Далі потрібно підготувати дані, з якими працюватиме агент. Джерел два: корпус і довідник. Вони різні за призначенням, і кожен обслуговує свій інструмент.

Корпус – набір текстових файлів з описовою інформацією. У ньому шукають за змістом: користувач формулює запит своїми словами, а інструмент пошуку знаходить відповідні записи.

Довідник – одна таблиця з точними значеннями. Тут нічого не шукають за змістом: інструмент знаходить рядок за ключем і повертає потрібні поля.

Дані вигадані, відповідність дійсності не перевіряється. Писати їх вручну не потрібно – згенеруйте будь-яким чат-ботом.

Генерація корпусу

Відкрийте чат-бот і надішліть запит такого вигляду (предметну область і структуру запису візьміть зі свого варіанта):

Згенеруй 12 анотацій книжок для навчального проєкту.
Дані вигадані.

Формат кожної анотації:

- перший рядок: назва книжки
- другий рядок: жанр і рівень складності
- далі 2-3 речення про зміст

Вимоги до набору:

- різні жанри: фантастика, технічна, побутова, історична
- різні рівні складності: легкий, середній, складний
- дві книжки зі схожими назвами
- одна книжка про сортування та алгоритми

Видай кожну анотацію окремим блоком, пронумеруй їх.

Збережіть кожну анотацію в окремий файл `.txt` у каталозі `data/corpus/`, кодування UTF-8. Разом має вийти 10–15 файлів.

Приклад файлу `data/corpus/b01.txt` :

Основи алгоритмів

Технічна література, складний рівень.

Класичний підручник із сортування, пошуку та оцінки складності.

Потребує базових знань математики.

Генерація довідника

Далі попросіть чат-бот скласти таблицю. Важливо передати йому список назв із корпусу – інакше ключі не збігатимуться:

```
Ось назви книжок:
Основи алгоритмів
Місто вічної ночі
Кулінарія для лінивих
[решта назв з вашого корпусу]

Склади на їх основі CSV-таблицю з колонками:
title, copies, loan_days, fine_per_day

Пояснення колонок:
- title: назва книжки, дослівно як у списку вище
- copies: кількість примірників, ціле число від 0 до 5
- loan_days: на скільки днів видають, 7 або 14
- fine_per_day: штраф за день прострочення, від 5 до 15 грн

Вимоги:
- назви мають збігатися зі списком посимвольно
- одна книжка зі списку має бути ВІДСУТНЯ в таблиці
- в однієї книжки copies = 0

Видай чистий CSV без пояснень і без markdown-розмітки.
```

Збережіть результат як `data/books.csv`. Назва файлу та перелік колонок для вашої предметної області вказані у файлі варіанта.

Приклад `data/books.csv`:

```
title,copies,loan_days,fine_per_day
Основи алгоритмів,1,7,10
Місто вічної ночі,3,14,5
Кулінарія для лінивих,0,14,5
```

Що перевірити після генерації

- у `data/corpus/` від 10 до 15 файлів;
- у `data/books.csv` від 15 до 25 рядків плюс заголовок;
- **назви в першій колонці CSV дослівно збігаються з першими рядками файлів корпусу** – розбіжність хоча б в одному символі зламає перехід від пошуку до довідника;
- у наборі є запис, присутній у корпусі та відсутній у довіднику;
- є запис із нульовим значенням;
- є дві схожі назви.

Останні три пункти потрібні для перевірки поведінки агента на некоректних запитах.

На цьому етапі структура проекту має виглядати так:

```
lab2/  
├─ venv/  
└─ data/  
    ├─ corpus/  
    │   ├─ b01.txt  
    │   ├─ b02.txt  
    │   └─ ...  
    └─ books.csv
```

Крок 1. Підключення до моделі

Усі налаштування агента зберігаються в одному файлі. Так їх легко змінювати, не шукаючи по всьому коду.

Створіть у каталозі проєкту файл `config.py`. У ньому треба вказати п'ять речей:

- адресу, за якою працює локальна Ollama;
- ключ доступу (Ollama його не перевіряє, але поле обов'язкове);
- назву моделі;
- ліміт кроків агента й температуру;
- системне повідомлення.

Назву моделі, ліміт кроків, температуру та текст системного повідомлення візьміть із текстового файлу свого варіанта – розділи «Параметри виклику» та «System message».

```
"""Налаштування агента."""

BASE_URL = "http://localhost:11434/v1"
API_KEY = "ollama"

MODEL = "qwen2.5:3b"
MAX_STEPS = 6
TEMPERATURE = 0

SYSTEM = """Ти помічник бібліотеки. Допмагаєш читачам знайти книжку
та дізнатися умови видачі.
Не вигадуй дані – усе, чого не знаєш, дізнавайся інструментом.
Не рахуй самостійно, для цього є окремий інструмент.
Якщо потрібного інструмента немає, чесно скажи про це.
Коли даних достатньо, дай коротку відповідь українською."""
```

У прикладі наведено значення для варіанта 1. Три лапки навколо системного повідомлення дозволяють писати текст у кілька рядків.

Перевірте, що модель завантажена й Ollama запущена:

```
ollama list
```

Структура проєкту на цьому етапі:

```
lab2/
├── venv/
├── data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └── ...
│   └── books.csv
└── config.py
```

Крок 2. Інструменти

Мовна модель не має доступу ні до файлів, ні до бази даних, ні до обчислень. Вона працює лише з текстом. Усе, що виходить за межі тексту, робить ваша програма – і саме для цього потрібні інструменти.

Інструмент – це звичайна функція Python. Вона читає дані з каталогу `data/`, обробляє їх і повертає результат **рядком**: модель уміє приймати тільки текст.

Далі в роботі ця функція буде викликатися не вами, а за вказівкою моделі: модель повідомить, який інструмент їй потрібен і з якими аргументами, а програма його викличе.

Створіть у каталозі проекту файл `tools.py`. Вихідний код трьох інструментів вашого варіанта наведено в текстовому файлі варіанта, у розділах «Інструмент 1», «Інструмент 2» та «Інструмент 3». Скопіюйте їх у `tools.py` без змін.

Зверніть увагу на два моменти в коді:

- шляхи до даних записані відносно каталогу проекту (`data/corpus/` , `data/books.csv`), тому запускати програми треба саме з нього;
- якщо інструмент нічого не знайшов, він повертає не порожній рядок, а повідомлення про це – моделі потрібен текст, який вона зможе прочитати.

Після копіювання перевірте роботу кожної функції окремо, без моделі. Для варіанта 1:

```
import tools

print(tools.search_books("фантастика"))
print(tools.book_info("Основи алгоритмів"))
print(tools.calculate("mul", 5, 10))
```

Якщо `search_books` нічого не знаходить, а `book_info` повідомляє, що книжки немає в довіднику, – найімовірніше, назви в корпусі та довіднику не збігаються.

Реєстр інструментів

Модель не може викликати функцію – вона лише повідомляє її назву рядком, наприклад `"book_info"`. Щоб за цим рядком знайти саму функцію, потрібен словник: ключ – назва, значення – функція.

Такий словник називають реєстром інструментів. Додайте його в кінець файлу `tools.py`:

```
TOOLS = {
    "search_books": search_books,
    "book_info": book_info,
    "calculate": calculate,
}
```

Праворуч ім'я функції пишеться **без дужок**. З дужками ви записали б у словник результат виклику, а потрібна сама функція.

Далі в програмі це працюватиме так:

<code>name = "book_info"</code>	= прийшло від моделі
<code>func = TOOLS[name]</code>	= дістали функцію зі словника
<code>result = func("Основи алгоритмів")</code>	= викликали

Загальна структура файлу `tools.py` :

```
import csv
import os

def search_books(query: str) -> str:
    ...

def book_info(title: str) -> str:
    ...

def calculate(operation: str, a: float, b: float) -> str:
    ...

TOOLS = {
    "search_books": search_books,
    "book_info": book_info,
    "calculate": calculate,
}
```

Назви інструментів у реєстрі мають дослівно збігатися з назвами з розділу «Опис для моделі» у файлі вашого варіанта.

Структура проєкту на цьому етапі:

```
lab2/
├─ venv/
├─ data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └── ...
│   └── books.csv
├─ config.py
└─ tools.py
```

Крок 3. Описи інструментів

Модель не бачить вашого коду. Вона не знає ні назв функцій, ні того, що вони роблять. Щоб вона могла скористатися інструментом, опис треба передати їй явно – у параметрі `tools` під час виклику:

```
response = client.chat.completions.create(
    model=MODEL,
    messages=messages,
    tools=SCHEMAS,      = ось сюди
)
```

Отримавши цей список, модель читає описи й вирішує, який інструмент підходить до поточного запиту. Тобто **вибір інструмента залежить від того, як ви його описали**, а не від того, як написана функція.

З чого складається опис

Опис одного інструмента – це вкладений словник. Розберемо його по шарах, від зовнішнього до внутрішнього.

Шар 1. Зовнішня обгортка. Однакова завжди:

```
{
  "type": "function",
  "function": { ... }
}
```

Шар 2. Усередині `"function"` – три ключі:

```
{
  "name": "назва функції",
  "description": "що робить і коли використовувати",
  "parameters": { ... }
}
```

`name` має дослівно збігатися з ключем у словнику `TOOLS`.

Шар 3. Усередині `"parameters"` – перелік аргументів:

```
{
  "type": "object",           = завжди саме так
  "properties": { ... },     = опис кожного аргумента
  "required": ["список обов'язкових"]
}
```

Шар 4. Усередині `"properties"` – по одному запису на аргумент:

```
"назва_аргумента": {
  "type": "string",          = string, number, integer або boolean
}
```

```

    "description": "що сюди передавати",
  }

```

Якщо аргумент приймає лише кілька конкретних значень, додайте `enum` – тоді модель обиратиме зі списку, а не вигадуватиме своє:

```

"currency": {
  "type": "string",
  "enum": ["USD", "EUR", "PLN"],
  "description": "Валюта, з якої переводимо",
}

```

Що писати в текстах

Два поля з описом – `description` функції та `description` кожного аргумента – це і є те, що читає модель. Решта полів технічні.

У `description` функції вкажіть: що вона робить, що повертає і **коли її використовувати**. Останнє пропускають найчастіше, а саме воно допомагає моделі обрати правильний інструмент.

У `description` аргумента вкажіть, у якому вигляді передавати значення. Без цього модель може передати в пошук усе питання користувача цілком.

Приклад

Створіть файл `schemas.py`. Нижче – приклад для двох вигаданих інструментів; описи для своїх трьох візьміть із розділів «Опис для моделі» у файлі варіанта.

```

"""Описи інструментів для моделі."""

SCHEMAS = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Повертає поточну температуру в місті. "
                          "Використовуй, коли користувач питає про погоду.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "Назва міста українською",
                    },
                },
                "required": ["city"],
            },
        },
    },
    {
        "type": "function",

```

```

    "function": {
        "name": "convert_currency",
        "description": "Переводить суму з однієї валюти в гривні "
                        "за поточним курсом. Використовуй ЗАВЖДИ "
                        "для переведення валют, не рахуй самостійно.",
        "parameters": {
            "type": "object",
            "properties": {
                "amount": {
                    "type": "number",
                    "description": "Сума, яку треба перевести",
                },
                "currency": {
                    "type": "string",
                    "enum": ["USD", "EUR", "PLN"],
                    "description": "Валюта, з якої переводимо",
                },
            },
            "required": ["amount", "currency"],
        },
    },
},
]

```

Ваш файл має містити три такі блоки – по одному на кожен інструмент.

Перевірте, що описів справді три і назви збігаються з реєстром:

```

from tools import TOOLS
from schemas import SCHEMAS

print(len(SCHEMAS))
print({s["function"]["name"] for s in SCHEMAS} == set(TOOLS))

```

Друга команда має вивести `True`. Якщо `False` – назва в описі не збігається з назвою в реєстрі, і агент не працюватиме.

Структура проєкту на цьому етапі:

```

lab2/
├─ venv/
├─ data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └─ ...
│   └─ books.csv
├─ config.py
├─ tools.py
└─ schemas.py

```

Крок 4. Диспетчер

Модель не викликає функцій. Вона повертає два рядки: назву інструмента і його аргументи. Наприклад:

```
name      = "book_info"
arguments = '{"title": "Основи алгоритмів"}'
```

Щоб перетворити це на реальний виклик, потрібен посередник – **диспетчер**. Він робить чотири речі:

1. знаходить функцію в реєстрі `TTOOLS` за назвою;
2. розбирає аргументи: вони приходять рядком, а функції потрібен словник;
3. викликає функцію;
4. повертає результат рядком, бо модель приймає лише текст.

Диспетчер – єдине місце в програмі, де можна щось перевірити перед виконанням. Модель може попросити що завгодно: назвати неіснуючий інструмент, передати зіпсований JSON або аргумент не того типу. Виконається тільки те, що диспетчер пропустив.

Чому помилки не можна просто дати впасти

Якщо диспетчер кине виключення, програма зупиниться. Ви проганяєте агента на п'яти питаннях, і на другому прогін обірветься – решта трьох навіть не запуститься.

Тому диспетчер **завжди повертає рядок і ніколи не кидає виключення**. Текст помилки потрапляє в контекст, і модель може прочитати його на наступному кроці й виправитися.

Код

Створіть файл `dispatcher.py` :

```
"""Виклик інструмента за назвою, отриманою від моделі."""
import json
from tools import TTOOLS

def call_tool(name: str, args_raw: str) -> str:
    = модель могла назвати інструмент, якого не існує
    if name not in TTOOLS:
        return (f"Помилка: інструмента '{name}' не існує. "
                f"Доступні: {', '.join(TTOOLS)}")
    try:
        args = json.loads(args_raw)          = рядок -> словник
        return str(TTOOLS[name](*args))      = * розпаковує словник в аргументи
    except json.JSONDecodeError:
        return "Помилка: аргументи не є коректним JSON."
    except TypeError as e:
        return f"Помилка в аргументах: {e}"
    except Exception as e:
        return f"Помилка виконання: {e}"
```

Конструкція `*args` перетворює словник `{"title": "Основи алгоритмів"}` на іменованний аргумент `title="Основи алгоритмів"`.

Зверніть увагу на перше повідомлення про помилку: воно не просто констатує проблему, а перелічує доступні інструменти. Модель прочитає цей список і зможе виправитися.

Цей файл однаковий для всіх варіантів – нічого підставляти не треба. Він працює з тим реєстром `T00LS`, який ви склали на кроці 2.

Перевірка

Додайте в кінець файлу й запустіть `python dispatcher.py`:

```
if __name__ == "__main__":
    print(call_tool("search_books", '{"query": "фантастика"}'))
    print(call_tool("delete_all", '{}'))
    print(call_tool("search_books", '{"query": фантастика}'))
    print(call_tool("search_books", '{"text": "фантастика"}'))
```

Перший виклик має повернути результат пошуку. Три наступні – повідомлення про помилку. Жоден не має завершитися виключенням.

Аргументи в цих прикладах записані так, як їх передає модель: у вигляді рядка з JSON усередині. Другий випадок – неіснуючий інструмент, третій – зіпсований JSON без лапок, четвертий – неправильна назва аргумента.

Структура проекту на цьому етапі:

```
lab2/
├── venv/
├── data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └── ...
│   └── books.csv
├── config.py
├── tools.py
├── schemas.py
└── dispatcher.py
```

Крок 5. Цикл агента

На цей момент готові всі складові:

- `config.py` – налаштування;
- `tools.py` – функції інструментів і реєстр;
- `schemas.py` – описи для моделі;
- `dispatcher.py` – виклик інструмента за назвою.

Лишилося з'єднати їх у працюючу систему. Цим займається цикл.

Навіщо потрібен саме цикл

На питання «знайди щось легке з фантастики» вистачить одного виклику інструмента. На питання «я затримав книжку на 5 днів, скільки штраф» потрібні три: знайти книжку, дізнатися ставку, помножити.

Скільки саме викликів знадобиться, наперед не знає ніхто – це залежить від питання й від того, що повернув попередній інструмент. Тому послідовність не програмують, а повторюють однакову операцію, поки модель не скаже, що готова відповісти.

Що робить один оберт циклу

1. Надіслати моделі всю історію повідомлень і описи інструментів.
2. Подивитися, що вона повернула:
 - якщо звичайний текст – це відповідь, виходимо з циклу;
 - якщо `tool_calls` – модель просить викликати інструмент.
3. Передати виклик диспетчеру.
4. Додати результат в історію з роллю `tool`.
5. Повернутися до пункту 1.

Ключова деталь: модель не має пам'яті. На кожному оберті їй передається **вся історія заново**, тому результати попередніх викликів мають накопичуватися у списку `messages`.

Ліміт кроків

Рішення зупинитися приймає модель. Вона може його не прийняти ніколи: викликати той самий інструмент по колу або нескінченно уточнювати.

Тому замість `while True` використовуємо `for` зі скінченною кількістю обертів. Якщо кроки скінчилися, а відповіді немає – зупиняємось примусово. Значення ліміту вже лежить у `config.MAX_STEPS`.

Лог

У циклі відбувається багато чого, а назовні видно лише кінцеву відповідь. Коли агент помилився, без логу зрозуміти причину буде неможливо.

Тому кожен крок записується: який інструмент викликано, з якими аргументами, що повернулося. Функція друку передається в `run_agent` параметром `log` – так її зручно замінити на запис у файл.

Код

Створіть файл `agent.py` :

```

"""Цикл агента."""
from openai import OpenAI

import config
from schemas import SCHEMAS
from dispatcher import call_tool

client = OpenAI(base_url=config.BASE_URL, api_key=config.API_KEY)

def run_agent(task, log):
    """task – питання користувача, log – функція друку.
    Повертає текст відповіді або None, якщо вичерпано ліміт."""

    = історія розмови: системне повідомлення і питання
    messages = [
        {"role": "system", "content": config.SYSTEM},
        {"role": "user", "content": task},
    ]

    log(f"\n{'=' * 60}")
    log(f"ЗАДАЧА: {task}")

    for step in range(1, config.MAX_STEPS + 1):

        = 1. запит до моделі: передаємо ВСЮ історію заново
        response = client.chat.completions.create(
            model=config.MODEL,
            messages=messages,
            tools=SCHEMAS,
            temperature=config.TEMPERATURE,
        )
        msg = response.choices[0].message

        log(f"\n--- крок {step} | prompt_tokens={response.usage.prompt_tokens}")

        = 2. немає tool_calls – модель дала відповідь, виходимо
        if not msg.tool_calls:
            log(f"ВІДПОВІДЬ: {msg.content}")
            return msg.content

        = 3. пропозицію моделі кладемо в історію
        messages.append(msg)

        = 4. виконуємо кожен запитаний виклик
        for call in msg.tool_calls:
            log(f"    виклик      : {call.function.name}({call.function.arguments})")

            result = call_tool(call.function.name, call.function.arguments)

            log(f"    результат: {result}")

```

```

    = 5. результат повертаємо моделі з роллю tool
    messages.append({
        "role": "tool",
        "tool_call_id": call.id,
        "content": result,
    })

    = сюди потрапляємо, якщо цикл дійшов до кінця без відповіді
    log("СТОП: вичерпано ліміт кроків")
    return None

```

Два місця, які варто розібрати окремо.

`messages.append(msg)` – у список додається сам об'єкт відповіді моделі, а не словник. Без цього рядка модель на наступному оберті не пам'ятатиме, що просила про виклик, і попросить знову.

`tool_call_id` – ідентифікатор конкретного виклику. Модель може попросити кілька інструментів одразу, і за цим полем вона розуміє, на який саме виклик прийшла відповідь.

Перевірка

Додайте в кінець файлу й запустіть `python agent.py` :

```

if __name__ == "__main__":
    run_agent("Знайди щось легке з фантастики", print)

```

Тут замість функції логування передається звичайний `print` – результат буде видно в консолі.

У виводі має бути видно принаймні один крок із рядками «виклик» і «результат», а потім відповідь.

Далі спробуйте багатокрокове питання свого варіанта (задача Z3). Агент має зробити кілька викликів поспіль, причому аргументи кожного наступного залежатимуть від результату попереднього.

Структура проекту на цьому етапі:

```

lab2/
├── venv/
├── data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └── ...
│   └── books.csv
├── config.py
├── tools.py
├── schemas.py
├── dispatcher.py
└── agent.py

```

Крок 6. Запуск на наборі питань

Агент уже працює, але запускати його по одному питанню незручно: щоб оцінити поведінку, потрібен прогін на всьому наборі одразу, а весь вивід має потрапити в один файл.

Цим займається окремий файл запуску. Він робить три речі: читає питання зі списку, проганяє кожне через `run_agent` і записує все, що відбувалося, у файл логу.

Файл із питаннями

Питання зберігаються окремо від коду, у форматі JSON. Так їх можна змінювати, не чіпаючи програму.

Створіть файл `tasks.json` і перенесіть у нього п'ять питань свого варіанта – три звичайні й дві пастки. Формат кожного запису:

```
[
  {
    "id": "Z1",
    "question": "Знайди щось легке з фантастики",
    "expected": "назва книжки з корпусу, жанр фантастика, рівень легкий"
  },
  {
    "id": "Z2",
    "question": "Знайди книжку про сортування і скажи, чи є вона в наявності",
    "expected": "назва книжки та кількість примірників"
  }
]
```

Поле `question` передається агенту як питання користувача. Поле `expected` агенту не передається – воно потрібне вам, щоб потім звірити з ним фактичну відповідь.

Функція логування

У попередньому кроці `run_agent` приймала параметр `log` і ми передавали туди звичайний `print`. Тепер замість нього підставимо функцію, яка друкує і пише у файл:

```
logfile = open("logs/run.txt", "w", encoding="utf-8")

def log(text):
    print(text)                = на екран, щоб бачити хід
    logfile.write(text + "\n") = і у файл, щоб потім розібрати
```

Саме заради цього функція друку передавалася параметром: тіло `run_agent` не змінюється, змінюється лише те, куди йде вивід.

Код

Створіть файл `run_all.py`:

```
"""Прогін агента на всіх питаннях із tasks.json."""
import json
from agent import run_agent
```

```

with open("tasks.json", encoding="utf-8") as f:
    tasks = json.load(f)

logfile = open("logs/run.txt", "w", encoding="utf-8")

def log(text):
    print(text)
    logfile.write(text + "\n")

for task in tasks:
    log(f"\n=== {task['id']} | очікується: {task['expected']}")
    run_agent(task["question"], log)

logfile.close()
print(f"\nОброблено задач: {len(tasks)}. Лог: logs/run.txt")

```

Перед кожною задачею в лог записується її номер і очікувана відповідь – так при розборі одразу видно, що саме мало вийти.

Запуск

```
python run_all.py
```

Прогін на п'яти питаннях триває приблизно 5–10 хвилин: на кожне питання припадає кілька звернень до моделі, а локальна модель відповідає повільно.

Запускайте один раз. Далі працюйте з файлом `logs/run.txt` – перезапустити агента, щоб щось передивитися, не потрібно.

Структура проєкту на цьому етапі:

```

lab2/
├── venv/
├── data/
│   ├── corpus/
│   │   ├── b01.txt
│   │   └── ...
│   └── books.csv
├── logs/
│   └── run.txt
├── config.py
├── tools.py
├── schemas.py
├── dispatcher.py
├── agent.py
├── run_all.py
└── tasks.json

```

Результат роботи

Таблиця поламок

Головний результат лабораторної. Відкрийте `logs/run.txt`, прочитайте його від початку до кінця й поррахуйте, скільки разів агент повівся неправильно. Одна задача може дати кілька поламок одночасно.

Поламка	Як упізнати в логу
Не викликав інструмент	на першому кроці немає рядка «виклик», одразу відповідь
Викликав не той інструмент	виклик є, але не той, що потрібен за умовою
Неправильні аргументи	у рядку «результат» слово «Помилка»
Зациклився	той самий виклик з тими самими аргументами двічі й більше
Не використав результат	інструмент повернув дані, у відповіді їх немає
Вигадав результат	у відповіді число або назва, яких немає в жодному «результаті»
Вичерпав ліміт кроків	у логу рядок «СТОП»

Остання колонка таблиці – загальна кількість по помилок по всіх п'яти задачах.

Таблиця результатів

Для кожної з п'яти задач:

Задача	Кроків	Викликів	Правильно (1/0)	Які поламки
Приклад завдання	3	2	1	

Оцінку 1 або 0 ставите самостійно, звіряючи відповідь агента з полем `expected`. Порівнювати рядки автоматично не можна: агент напише «штраф становитиме 50 гривень» замість «50 грн».

Для задач-пасток оцінка 1 ставиться, якщо агент чесно повідомив, що не може виконати запит.

Розбір одного провалу

Оберіть найпоказовіший випадок і наведіть:

- номер задачі й текст питання;
- повний фрагмент логу без скорочень;
- яка відповідь мала бути;

- чому, на вашу думку, агент повівся саме так.

Пояснення «модель слабка» не приймається. Розгляньте конкретні причини: опис інструмента не давав зрозуміти, коли його застосовувати; модель не побачила, що потрібен другий крок; результат повернувся у вигляді, який модель не змогла використати.

Відповіді на питання

По одному-два речення:

1. Скільки задач із п'яти агент розв'язав правильно?
2. Яка поламка трапилася найчастіше?
3. Чи був випадок, коли агент вигдав результат, не викликавши інструмент?
4. Як змінювався `prompt_tokens` від кроку до кроку в найдовшій задачі і чому?
5. Чи пройшов агент задачі-пастки?

Звіт

Оформлюється у форматі PDF. Структура:

1. Титульна сторінка.
2. Мета роботи, номер варіанта, предметна область.
3. Склад даних: скільки файлів у корпусі, скільки рядків у довіднику.
4. Описи трьох інструментів, які ви склали для моделі, – розділ `SCHEMAS` із файлу `schemas.py`.
5. Таблиця результатів по п'яти задачах.
6. Таблиця поламок.
7. Розбір одного провалу.
8. Відповіді на питання.
9. Висновок: що виявилось найважчим і яка поламка здається вам найнебезпечнішою для реальної системи.

Що здається

Архів або репозиторій із такою структурою:

```
lab2/
├─ report.pdf
├─ data/
│   └─ corpus/
│       └─ books.csv
├─ logs/
│   └─ run.txt
├─ config.py
├─ tools.py
├─ schemas.py
├─ dispatcher.py
├─ agent.py
├─ run_all.py
└─ tasks.json
```

Каталог `venv/` до архіву не включається.

Робота без файлу `logs/run.txt` не приймається: лог є єдиним підтвердженням того, що прогін відбувся, і єдиним джерелом для таблиці поламок.